

Agentic AI Mastery Syllabus

We are dividing the work to master "Agentic AI".

- Aadi focuses on **Capabilities** (Tools/Infrastructure).
- Jeevitha focuses on **Cognition** (Reasoning/Logic).

We will learn separately and then teach each other what we learned.

Aadi's Path: The Toolmaker (Infrastructure & Capabilities)

Your Goal: "Make the AI capable of doing anything on the computer."

1. Python Integration (The Basics)

- **What to learn:** How to write Python functions that interact with the outside world.
- **Key Library:** `requests` (for APIs).
- **Project Idea:** Build a tool that fetches the Weather from OpenWeatherMap API.
- **Why:** The AI is stuck in a box. You build the windows and doors.

2. File Handling (The Data)

- **What to learn:** How to read PDFs, Excel sheets, and Text files using Python.
- **Key Libraries:** `pypdf` , `pandas` , `open()` .
- **Project Idea:** Build a tool called `read_resume` that extracts text from a PDF so the Agent can interview a candidate.

3. Database Interaction (Memory)

- **What to learn:** How to save data so it isn't lost when the chat ends.
- **Key Library:** `sqlite3` (Built into Python).
- **Project Idea:** A tool called `save_note` that writes a user's reminder to a database file.

4. Vector Databases (Advanced Memory)

- **What to learn:** How to store "meanings" instead of just text (RAG - Retrieval Augmented Generation).
- **Key Library:** `ChromaDB` (runs locally).
- **Why:** So the AI can "read" a 100-page book and answer questions about it without crashing.

Jeevitha's Path: The Architect (Logic & Reasoning)

Your Goal: "Make the AI smart enough to use Aadi's tools correctly."

1. Prompt Engineering (The Foundation)

- **What to learn:** How to write System Prompts that force the AI to behave.
- **Concept:** "Chain of Thought" prompting.
- **Task:** Fix the "Capital of France" bug. Tell the AI: *"Only use the Time Tool if the user asks for the time. Otherwise, answer directly."*

2. LangGraph Flows (The Workflow)

- **What to learn:** How to build complex decision trees.
- **Key Concepts:** Nodes, Edges, Conditional Edges.
- **Challenge:** Build a flow where the AI searches Google -> Checks if the answer is good -> If not, searches again (Looping).

3. Structured Output (Reliability)

- **What to learn:** How to force the AI to return JSON instead of text.
- **Why:** If Aadi's tool expects a number but the AI replies "The answer is five", the code breaks. You need to force the AI to output `{ "answer": 5 }`.

Learning Resources (Free)

For Aadi (Tools & Python)

- **Real Python (Website):** Best tutorials for `requests` and `pandas`.
- **LangChain Documentation (Tools Section):** Read about "Custom Tools".

For Jeevitha (Logic & Agents)

- **DeepLearning.AI (Course):** "Functions, Tools and Agents with LangChain" (Free short course).
- **LangGraph Tutorials (YouTube):** Search for "LangGraph crash course".

For Both

- **Ollama Library:** Experiment with different models (`mistral` , `gemma`) to see which one is smarter for our specific tasks.